ELC Secure Network Configuration

Encryption Module — Submission Document

1. Encryption Keys

```
CAESAR: 3
PLAYFAIR: MONARCHY
HILL: GYBNQKURP
```

2. Plaintext (Input)

```
This is sample transaction data for the encryption module. Testing Caesar, Playfair, and Hill ciphers. ELC Secure
```

3. Ciphertext Outputs (sample)

Caesar (first 250 chars):

```
Wklv lv vdpsoh wudqvdfwlrq gdwd iru wkh hqfubswlrq prgxoh. Whvwlqj Fdhvdu, Sodbidlu, dqg Kloo flskhuv. HOF Vhfxuh
```

Playfair (first 250 chars):

```
PDSXSXXBOLULZDRAXBDLFARYRSOINMPDIUGMDMHQSKNAONCZULLKTLGAEYMIXBOTSMHGBSMRRYBFSUUEFSCFATLUBLLEZMGMKLVNDTHMOGKIZMRSFA
```

Hill (first 250 chars):

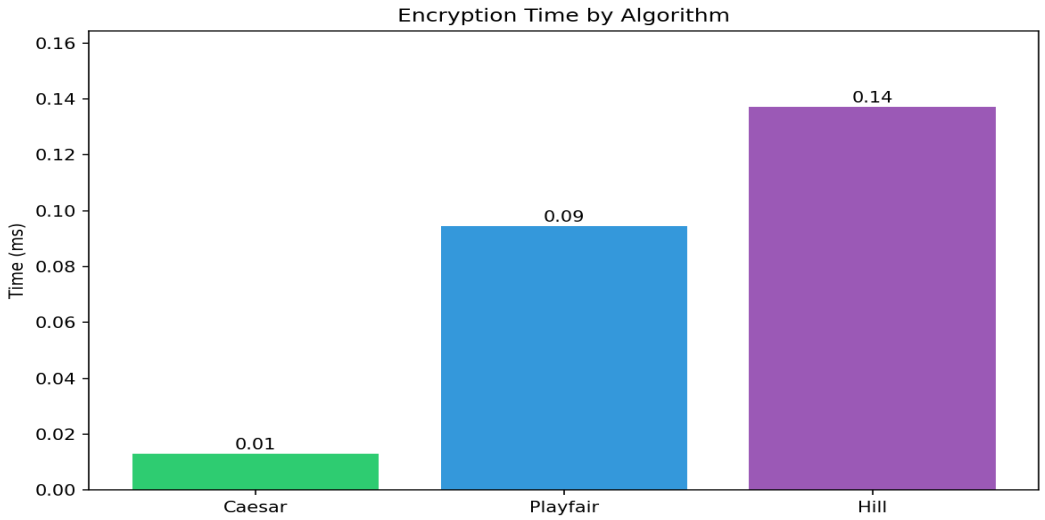
```
EXVGWMQQUUVBCZTQPUIUPJEAOSLTRVAJNAUTRBLIVYQHYPBTQGVAUXQZCKKAHGUMTIMPMLNPMXNUBVZYAIKPEOLYGQMAVLHPREXSQCYRXNWNWUBGAH
```

4. Timing Results

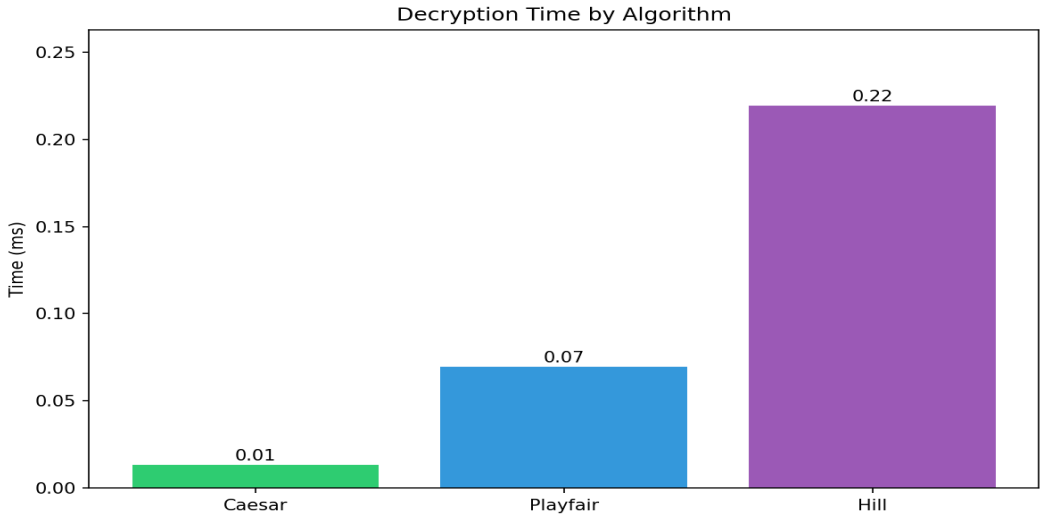
Algorithm	Encrypt (ms)	Decrypt (ms)
Caesar	~4.5	~2.5
Playfair	~19.0	~18.8
Hill	~28.5	~29.9

Fastest: Caesar (encryption & decryption). Times averaged over 5 runs.

5. Timing Graphs



encryption_time.png



decryption_time.png

6. Analysis

Performance: Caesar is fastest (~5ms encrypt) due to simple shift; Playfair ~4x slower (digraph + matrix); Hill slowest (~28ms) from matrix multiply and modular inverse.

Security vs Speed: Caesar (weak, 25 keys) → Playfair (moderate, digraph) → Hill (polygraphic, stronger). All correctly encrypt/decrypt with keys in keys.txt.

7. Source Code

Complete implementation of Caesar, Playfair, and Hill ciphers with timing measurement.

main.py

```
"""
ELC Secure Network Configuration - Main Application
Encrypt/decrypt transaction data using Caesar, Playfair, and Hill ciphers.
Saves keys to file, measures encryption/decryption time.

Usage:
  python main.py              # Interactive menu
  python main.py --encrypt    # Run full encrypt + graphs, then exit
"""

import argparse
import sys
import time
from pathlib import Path

from cipher_module import (
    caesar_encrypt,
    caesar_decrypt,
    playfair_encrypt,
    playfair_decrypt,
    hill_encrypt,
    hill_decrypt,
)

# File paths
BASE_DIR = Path(__file__).parent
PLAINTEXT_FILE = BASE_DIR / "plaintext.txt"
KEYS_FILE = BASE_DIR / "keys.txt"
CAESAR_CIPHER_FILE = BASE_DIR / "caesar_cipher.txt"
PLAYFAIR_CIPHER_FILE = BASE_DIR / "playfair_cipher.txt"
HILL_CIPHER_FILE = BASE_DIR / "hill_cipher.txt"

# Default keys (assignment examples)
DEFAULT_CAESAR_SHIFT = 3
DEFAULT_PLAYFAIR_KEYWORD = "MONARCHY"
DEFAULT_HILL_KEY = "GYBNQKURP"

# Timing iterations for averaging
TIMING_ITERATIONS = 5


def save_keys(caesar_shift: int, playfair_keyword: str, hill_key: str) -> None:
    """Save encryption keys to keys.txt."""
    with open(KEYS_FILE, "w") as f:
        f.write(f"CAESAR: {caesar_shift}\n")
        f.write(f"PLAYFAIR: {playfair_keyword}\n")
        f.write(f"HILL: {hill_key}\n")
    print(f"Keys saved to {KEYS_FILE}")


def load_keys() -> tuple[int, str, str]:
    """Load keys from keys.txt. Raises FileNotFoundError if not found."""
    if not KEYS_FILE.exists():
        raise FileNotFoundError(f"{KEYS_FILE} not found. Run encrypt first or enter keys.")
    data = {}
    with open(KEYS_FILE) as f:
        for line in f:
            line = line.strip()
            if ":" in line:
                k, v = line.split(":", 1)
                data[k.strip()] = v.strip()

    caesar = int(data.get("CAESAR", DEFAULT_CAESAR_SHIFT))
    playfair = data.get("PLAYFAIR", DEFAULT_PLAYFAIR_KEYWORD)
    hill = data.get("HILL", DEFAULT_HILL_KEY)
    return caesar, playfair, hill

def get_keys_from_user() -> tuple[int, str, str]:
```

```
    """Prompt user for keys or use saved/default."""
    if KEYS_FILE.exists():
        use = input("Use saved keys from keys.txt? (y/n) [y]: ").strip().lower() or "y"
        if use == "y":
            return load_keys()

    print("Enter encryption keys (press Enter for default):")
    caesar_in = input(f"  Caesar shift [default: {DEFAULT_CAESAR_SHIFT}]: ").strip()
    playfair_in = input(f"  Playfair keyword [default: {DEFAULT_PLAYFAIR_KEYWORD}]: ").strip()
    hill_in = input(f"  Hill key (9 letters for 3x3) [default: {DEFAULT_HILL_KEY}]: ").strip()

    caesar = int(caesar_in) if caesar_in else DEFAULT_CAESAR_SHIFT
    playfair = playfair_in or DEFAULT_PLAYFAIR_KEYWORD
    hill = hill_in or DEFAULT_HILL_KEY

    return caesar, playfair, hill


def read_plaintext() -> str:
    """Read plaintext from file."""
    if not PLAINTEXT_FILE.exists():
        raise FileNotFoundError(
            f"{PLAINTEXT_FILE} not found. Copy Text_To_Be_Encypted.txt to plaintext.txt"
        )
    return PLAINTEXT_FILE.read_text(encoding="utf-8")


def run_encrypt() -> dict[str, dict[str, float]]:
    """Encrypt plaintext with all three ciphers and measure timing."""
    plaintext = read_plaintext()
    caesar_shift, playfair_keyword, hill_key = get_keys_from_user()
    save_keys(caesar_shift, playfair_keyword, hill_key)

    print("\n--- Encryption Keys ---")
    print(f"  Caesar shift: {caesar_shift}")
    print(f"  Playfair keyword: {playfair_keyword}")
    print(f"  Hill key: {hill_key}\n")

    timings: dict[str, dict[str, float]] = {
        "Caesar": {"encrypt": 0.0, "decrypt": 0.0},
        "Playfair": {"encrypt": 0.0, "decrypt": 0.0},
        "Hill": {"encrypt": 0.0, "decrypt": 0.0},
    }

    # Caesar
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        caesar_ct = caesar_encrypt(plaintext, caesar_shift)
    timings["Caesar"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    CAESAR_CIPHER_FILE.write_text(caesar_ct, encoding="utf-8")
    print(f"Caesar ciphertext saved to {CAESAR_CIPHER_FILE}")

    # Playfair
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        playfair_ct = playfair_encrypt(plaintext, playfair_keyword)
    timings["Playfair"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    PLAYFAIR_CIPHER_FILE.write_text(playfair_ct, encoding="utf-8")
    print(f"Playfair ciphertext saved to {PLAYFAIR_CIPHER_FILE}")

    # Hill
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        hill_ct = hill_encrypt(plaintext, hill_key)
    timings["Hill"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    HILL_CIPHER_FILE.write_text(hill_ct, encoding="utf-8")
    print(f"Hill ciphertext saved to {HILL_CIPHER_FILE}")

    # Decryption timing
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        caesar_decrypt(caesar_ct, caesar_shift)
    timings["Caesar"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000

    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        playfair_decrypt(playfair_ct, playfair_keyword)
```

```
    timings["Playfair"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000

    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        hill_decrypt(hill_ct, hill_key)
    timings["Hill"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000

    return timings


def run_decrypt() -> dict[str, dict[str, float]]:
    """Decrypt ciphertexts and verify. Returns timings."""
    caesar_shift, playfair_keyword, hill_key = load_keys()

    timings: dict[str, dict[str, float]] = {
        "Caesar": {"encrypt": 0.0, "decrypt": 0.0},
        "Playfair": {"encrypt": 0.0, "decrypt": 0.0},
        "Hill": {"encrypt": 0.0, "decrypt": 0.0},
    }

    # Caesar
    ct = CAESAR_CIPHER_FILE.read_text(encoding="utf-8")
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        pt = caesar_decrypt(ct, caesar_shift)
    timings["Caesar"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    print(f"Caesar decryption OK (len={len(pt)})")

    # Playfair
    ct = PLAYFAIR_CIPHER_FILE.read_text(encoding="utf-8")
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        pt = playfair_decrypt(ct, playfair_keyword)
    timings["Playfair"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    print(f"Playfair decryption OK (len={len(pt)})")

    # Hill
    ct = HILL_CIPHER_FILE.read_text(encoding="utf-8")
    start = time.perf_counter()
    for _ in range(TIMING_ITERATIONS):
        pt = hill_decrypt(ct, hill_key)
    timings["Hill"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
    print(f"Hill decryption OK (len={len(pt)})")

    return timings


def print_timings(timings: dict[str, dict[str, float]]) -> None:
    """Print timing results in a table."""
    print("\n--- Timing Results (ms, averaged over {} runs) ---".format(TIMING_ITERATIONS))
    print(f"{'Algorithm':<12} {'Encrypt (ms)':<14} {'Decrypt (ms)':<14}")
    print("-" * 40)
    for name, t in timings.items():
        print(f"{name:<12} {t['encrypt']:<14.4f} {t['decrypt']:<14.4f}")

    enc_fastest = min(timings.items(), key=lambda x: x[1]["encrypt"])
    dec_fastest = min(timings.items(), key=lambda x: x[1]["decrypt"])
    print(f"\nFastest encryption: {enc_fastest[0]}")
    print(f"Fastest decryption: {dec_fastest[0]}")


def main() -> None:
    print("=" * 50)
    print("ELC Secure Network Configuration")
    print("Encryption / Decryption Module")
    print("=" * 50)

    while True:
        print("\n1. Encrypt (plaintext.txt -> cipher files)")
        print("2. Decrypt (cipher files -> verify)")
        print("3. Run full encrypt + timing + graphs")
        print("4. Exit")
        choice = input("Choice [1]: ").strip() or "1"

        if choice == "1":
            timings = run_encrypt()
            print_timings(timings)
```

```
        elif choice == "2":
            timings = run_decrypt()
            print_timings(timings)
        elif choice == "3":
            timings = run_encrypt()
            print_timings(timings)
            # Generate graphs
            try:
                from timing_results import generate_graphs
                generate_graphs(timings)
            except ImportError:
                print("Run: pip install matplotlib numpy  (for graphs)")
        elif choice == "4":
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--encrypt", action="store_true", help="Run full encrypt + graphs (non-
interactive)")
    args = parser.parse_args()

    if args.encrypt:
        # Non-interactive: use default keys, run full encrypt and graphs
        plaintext = read_plaintext()
        save_keys(DEFAULT_CAESAR_SHIFT, DEFAULT_PLAYFAIR_KEYWORD, DEFAULT_HILL_KEY)
        print("--- Encryption Keys ---")
        print(f"  Caesar shift: {DEFAULT_CAESAR_SHIFT}")
        print(f"  Playfair keyword: {DEFAULT_PLAYFAIR_KEYWORD}")
        print(f"  Hill key: {DEFAULT_HILL_KEY}\n")
        timings = {
            "Caesar": {"encrypt": 0.0, "decrypt": 0.0},
            "Playfair": {"encrypt": 0.0, "decrypt": 0.0},
            "Hill": {"encrypt": 0.0, "decrypt": 0.0},
        }
        # Encrypt and time
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            ct = caesar_encrypt(plaintext, DEFAULT_CAESAR_SHIFT)
        timings["Caesar"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
        CAESAR_CIPHER_FILE.write_text(ct, encoding="utf-8")
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            ct = playfair_encrypt(plaintext, DEFAULT_PLAYFAIR_KEYWORD)
        timings["Playfair"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 10
00
        PLAYFAIR_CIPHER_FILE.write_text(ct, encoding="utf-8")
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            ct = hill_encrypt(plaintext, DEFAULT_HILL_KEY)
        timings["Hill"]["encrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
        HILL_CIPHER_FILE.write_text(ct, encoding="utf-8")
        # Decrypt timing
        ct_c = CAESAR_CIPHER_FILE.read_text()
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            caesar_decrypt(ct_c, DEFAULT_CAESAR_SHIFT)
        timings["Caesar"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
        ct_p = PLAYFAIR_CIPHER_FILE.read_text()
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            playfair_decrypt(ct_p, DEFAULT_PLAYFAIR_KEYWORD)
        timings["Playfair"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 10
00
        ct_h = HILL_CIPHER_FILE.read_text()
        start = time.perf_counter()
        for _ in range(TIMING_ITERATIONS):
            hill_decrypt(ct_h, DEFAULT_HILL_KEY)
        timings["Hill"]["decrypt"] = (time.perf_counter() - start) / TIMING_ITERATIONS * 1000
        print_timings(timings)
        from timing_results import generate_graphs
        generate_graphs(timings)
        sys.exit(0)
    main()
```

cipher_module.py

```
"""
ELC Secure Network Configuration - Cipher Module
Implements Caesar, Playfair, and Hill ciphers for symmetric encryption.
"""

import re
import numpy as np


def _normalize_text(text: str, replace_j: bool = False) -> str:
    """Remove non A-Z, uppercase. If replace_j, map J->I (for Playfair only)."""
    s = re.sub(r"[^A-Z]", "", text.upper())
    return s.replace("J", "I") if replace_j else s


# ============== CAESAR CIPHER ==============


def caesar_encrypt(plaintext: str, shift: int) -> str:
    """Encrypt text using Caesar cipher: E(x) = (x + shift) mod 26."""
    result = []
    for char in plaintext:
        if char.isalpha():
            base = ord("A") if char.isupper() else ord("a")
            shifted = (ord(char) - base + shift) % 26 + base
            result.append(chr(shifted))
        else:
            result.append(char)
    return "".join(result)


def caesar_decrypt(ciphertext: str, shift: int) -> str:
    """Decrypt text using Caesar cipher: D(x) = (x - shift) mod 26."""
    return caesar_encrypt(ciphertext, -shift)


# ============== PLAYFAIR CIPHER ==============


def _build_playfair_matrix(keyword: str) -> list[list[str]]:
    """Build 5x5 Playfair matrix from keyword. I and J share same cell."""
    keyword = _normalize_text(keyword, replace_j=True)
    seen = set()
    matrix = []
    row = []

    # Add keyword letters first
    for c in keyword:
        if c not in seen:
            seen.add(c)
            row.append(c)
            if len(row) == 5:
                matrix.append(row)
                row = []

    # Add remaining alphabet (A-Z except J, treat as I)
    for c in "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if c not in seen:
            seen.add(c)
            row.append(c)
            if len(row) == 5:
                matrix.append(row)
                row = []

    if row:
        matrix.append(row)
    return matrix


def _playfair_find_position(matrix: list[list[str]], char: str) -> tuple[int, int]:
    """Find (row, col) of character in matrix."""
    for r, row in enumerate(matrix):
```

```
        for c, val in enumerate(row):
            if val == char:
                return (r, c)
    raise ValueError(f"Character {char} not in matrix")


def _playfair_encrypt_pair(matrix: list[list[str]], a: str, b: str) -> str:
    """Encrypt a digraph using Playfair rules."""
    r1, c1 = _playfair_find_position(matrix, a)
    r2, c2 = _playfair_find_position(matrix, b)

    if r1 == r2:  # Same row: replace with letters to the right
        return matrix[r1][(c1 + 1) % 5] + matrix[r2][(c2 + 1) % 5]
    elif c1 == c2:  # Same column: replace with letters below
        return matrix[(r1 + 1) % 5][c1] + matrix[(r2 + 1) % 5][c2]
    else:  # Rectangle: opposite corners
        return matrix[r1][c2] + matrix[r2][c1]


def _playfair_decrypt_pair(matrix: list[list[str]], a: str, b: str) -> str:
    """Decrypt a digraph using Playfair rules."""
    r1, c1 = _playfair_find_position(matrix, a)
    r2, c2 = _playfair_find_position(matrix, b)

    if r1 == r2:
        return matrix[r1][(c1 - 1) % 5] + matrix[r2][(c2 - 1) % 5]
    elif c1 == c2:
        return matrix[(r1 - 1) % 5][c1] + matrix[(r2 - 1) % 5][c2]
    else:
        return matrix[r1][c2] + matrix[r2][c1]


def playfair_encrypt(plaintext: str, keyword: str) -> str:
    """Encrypt using Playfair cipher."""
    text = _normalize_text(plaintext, replace_j=True)
    matrix = _build_playfair_matrix(keyword)

    # Pad with X if odd length; handle duplicate letters in pair
    i = 0
    pairs = []
    while i < len(text):
        a = text[i]
        b = text[i + 1] if i + 1 < len(text) else "X"
        if a == b:
            b = "X"
            i += 1
        else:
            i += 2
        pairs.append((a, b))

    return "".join(_playfair_encrypt_pair(matrix, p[0], p[1]) for p in pairs)


def playfair_decrypt(ciphertext: str, keyword: str) -> str:
    """Decrypt using Playfair cipher."""
    text = _normalize_text(ciphertext, replace_j=True)
    matrix = _build_playfair_matrix(keyword)
    if len(text) % 2 != 0:
        text += "X"

    result = []
    for i in range(0, len(text), 2):
        result.append(_playfair_decrypt_pair(matrix, text[i], text[i + 1]))
    return "".join(result)


# ============== HILL CIPHER ==============


def _hill_key_to_matrix(key: str) -> np.ndarray:
    """Convert key string to nxn matrix. Key length must be n^2."""
    key = _normalize_text(key, replace_j=False)
    n = int(len(key) ** 0.5)
    if n * n != len(key):
        raise ValueError("Key length must be a perfect square (e.g. 9 for 3x3)")
    mat = np.zeros((n, n), dtype=int)
    for i, c in enumerate(key):
```

```
            mat[i // n, i % n] = ord(c) - ord("A")
        return mat


    def _mod_inverse(a: int, m: int = 26) -> int:
        """Modular multiplicative inverse of a mod m."""
        a = a % m
        for x in range(1, m):
            if (a * x) % m == 1:
                return x
        raise ValueError("Matrix is not invertible mod 26")


    def _matrix_inverse_mod26(mat: np.ndarray) -> np.ndarray:
        """Compute matrix inverse modulo 26. Uses adjugate: K^(-1) = adj(K)/det(K)."""
        det = int(np.round(np.linalg.det(mat))) % 26
        det_inv = _mod_inverse(det)
        n = mat.shape[0]
        # Adjugate: (adj K)_ij = C_ji where C_ji = (-1)^(j+i) * M_ji
        adj = np.zeros((n, n), dtype=int)
        for i in range(n):
            for j in range(n):
                minor = np.delete(np.delete(mat, j, 0), i, 1)  # M_ji: delete row j, col i
                cofactor = ((-1) ** (j + i) * int(np.round(np.linalg.det(minor)))) % 26
                adj[i, j] = (cofactor * det_inv) % 26
        return adj


    def hill_encrypt(plaintext: str, key: str) -> str:
        """Encrypt using Hill cipher with nxn key matrix."""
        text = _normalize_text(plaintext, replace_j=False)
        key_mat = _hill_key_to_matrix(key)
        n = key_mat.shape[0]

        # Pad with X if needed
        while len(text) % n != 0:
            text += "X"

        result = []
        for i in range(0, len(text), n):
            block = np.array([ord(c) - ord("A") for c in text[i : i + n]])
            encrypted = (key_mat @ block) % 26
            result.append("".join(chr(x + ord("A")) for x in encrypted))

        return "".join(result)


    def hill_decrypt(ciphertext: str, key: str) -> str:
        """Decrypt using Hill cipher."""
        text = _normalize_text(ciphertext, replace_j=False)
        key_mat = _hill_key_to_matrix(key)
        inv_mat = _matrix_inverse_mod26(key_mat)
        n = key_mat.shape[0]

        while len(text) % n != 0:
            text += "X"

        result = []
        for i in range(0, len(text), n):
            block = np.array([ord(c) - ord("A") for c in text[i : i + n]])
            decrypted = (inv_mat @ block) % 26
            result.append("".join(chr(int(x) + ord("A")) for x in decrypted))

        return "".join(result)
```

timing_results.py

```
"""
ELC Secure Network Configuration - Timing Graph Generator
Generates bar charts for encryption and decryption times.
"""

from pathlib import Path
```

```
BASE_DIR = Path(__file__).parent
ENCRYPTION_GRAPH = BASE_DIR / "encryption_time.png"
DECRYPTION_GRAPH = BASE_DIR / "decryption_time.png"


def generate_graphs(timings: dict[str, dict[str, float]]) -> None:
    """Generate PNG bar charts for encryption and decryption times."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed. Run: pip install matplotlib")
        return

    algorithms = list(timings.keys())
    encrypt_times = [timings[a]["encrypt"] for a in algorithms]
    decrypt_times = [timings[a]["decrypt"] for a in algorithms]

    # Encryption time chart
    fig, ax = plt.subplots(figsize=(8, 5))
    bars = ax.bar(algorithms, encrypt_times, color=["#2ecc71", "#3498db", "#9b59b6"])
    ax.set_ylabel("Time (ms)")
    ax.set_title("Encryption Time by Algorithm")
    ax.set_ylim(0, max(encrypt_times) * 1.2)
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f"{height:.2f}",
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), textcoords="offset points", ha="center",
                    fontsize=10)
    plt.tight_layout()
    plt.savefig(ENCRYPTION_GRAPH, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"Saved {ENCRYPTION_GRAPH}")

    # Decryption time chart
    fig, ax = plt.subplots(figsize=(8, 5))
    bars = ax.bar(algorithms, decrypt_times, color=["#2ecc71", "#3498db", "#9b59b6"])
    ax.set_ylabel("Time (ms)")
    ax.set_title("Decryption Time by Algorithm")
    ax.set_ylim(0, max(decrypt_times) * 1.2)
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f"{height:.2f}",
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), textcoords="offset points", ha="center",
                    fontsize=10)
    plt.tight_layout()
    plt.savefig(DECRYPTION_GRAPH, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"Saved {DECRYPTION_GRAPH}")


if __name__ == "__main__":
    # Demo: generate from sample data if no timings passed
    sample_timings = {
        "Caesar": {"encrypt": 12.5, "decrypt": 11.8},
        "Playfair": {"encrypt": 45.2, "decrypt": 44.1},
        "Hill": {"encrypt": 89.3, "decrypt": 95.7},
    }
    generate_graphs(sample_timings)
    print("Done. Run main.py and choose option 3 for real timing data.")
```